VITyarthi – Build Your Own Project

STUDENT MANAGEMENT SYSTEM

Project Report

Author
Vishal Pandey

Program
B.Tech CSE – 1st Semester

Institute
VIT Bhopal University

Prepared for VITyarthi Project Submission

1. Introduction

The Student Management System is a small Python console application built around everyday student activities. It brings basic profile information, marks, attendance, and study tasks into one menu-driven program.

The project is intentionally straightforward. The focus is on applying programming fundamentals to a familiar problem: functions, modules, loops, conditions, lists, validation, exception handling, calculations, and file handling.

The application is divided into separate Python files. The main program connects these modules and gives the user four working areas plus an exit option.

The VITyarthi instructions ask students to identify a meaningful problem, design a technical solution, implement it using course concepts, and demonstrate understanding through documentation and evaluation. This report follows that structure while describing the uploaded implementation without changing it.

2. Problem Statement

Students regularly deal with small academic activities such as maintaining profile details, checking marks, calculating attendance, and keeping study tasks. Handling these activities separately can be inconvenient. This project provides one simple console application for these activities.

It is not intended to replace a full institutional information system. It is a first-semester programming project demonstrating how a real-world idea can be divided into manageable modules.

3. Objectives

• Build a simple student-oriented console application using Python.

• Practice modular programming by separating features into individual files.

• Collect and display basic student profile information.

• Calculate marks, percentage, and grade for three subjects.

• Calculate attendance and check it against the program's 75% condition.

• Provide a simple study-task list.

• Apply validation, exception handling, calculations, and file handling.

• Document the project clearly for academic evaluation.

4. Scope

The current scope covers student profile management, marks management, attendance management, study tasks, calculations, validation, and file operations. The application remains command-line based and uses simple file handling rather than a database.

The current project does not provide a graphical interface, online access, login system, database, or network service. These are treated as future possibilities, not current features.

5. Target Users

• Students learning introductory Python and modular programming.

• Students who want a small academic console application.

• Teachers and evaluators reviewing fundamental Python concepts.

6. Functional Requirements

Module	Current functionality
Student Profile	Accepts name, roll number, branch, and year; saves and displays the profile.
Marks Manager	Accepts three subject marks, validates input, calculates total/percentage, and shows grade.
Attendance	Accepts total and attended classes, calculates percentage, and reports status.
Study Tasks	Adds and displays tasks during the current task-manager session.
Utilities	Provides calculation, validation, and file-management functions.

7. Non-Functional Requirements

These describe reasonable quality expectations for this small console application. They should not be interpreted as claims of advanced production-level guarantees.

Requirement	Project interpretation
Usability	Menu and prompts should be understandable to a student using the console.
Reliability	Valid inputs should produce predictable results and existing validation should respond to invalid cases.
Maintainability	Separate files keep major functionality easier to understand and update.
Error Handling	Marks input handles invalid numeric/out-of-range values; attendance checks attended classes against total clas
Resource Efficiency	The program uses simple calculations, lists, and text-file operations.
Scalability	The present design suits a small console project; larger storage would need a different approach.

8. Technical Expectations and Context

The VITyarthi instructions call for modular implementation, validation and error handling, documentation, version control, and, for coding projects, 5–10 meaningful modules/classes/files. The uploaded project contains eight Python source files and separates the main areas of functionality.

The project uses functions and modules rather than classes. That choice fits an introductory Python project and keeps the implementation approachable.

9. Technologies Used

• Python 3.x

• Functions and modules

• Loops and conditional statements

• Lists and exception handling

• File handling and console input/output

10. System Architecture

The architecture is intentionally simple. main.py is the entry point and routes the user's choice to the appropriate feature module. Supporting modules provide calculations, validation, and file operations.

User Console input			
main.py Menu and control flow			
student.py Student Profile	marks2.py Marks	attendance.py Attendance	tasks.py Study Tasks
calculation.py Calculations	validation.py Validation	file_manager.py File Operations	student.txt Profile storage

11. Module Responsibilities

• **main.py:** displays the main menu and calls the selected function.

• **student.py:** collects profile details and saves them.

• **marks2.py:** accepts marks, calculates percentage, and displays a grade.

• **attendance.py:** calculates attendance and checks the 75% condition.

• **tasks.py:** manages the task list during the current session.

• **calculation.py:** contains average, percentage, and maximum functions.

• **validation.py:** checks marks between 0 and 100.

• **file_manager.py:** provides save and load operations.

12. Workflow Diagram

The workflow below represents the menu interaction already present in the project.

START
Display Student Management System menu
User selects an option
1 Profile | 2 Marks | 3 Attendance | 4 Tasks | 5 Exit
Execute selected module
Return to main menu
5 selected → END

13. Use Case Diagram

The primary actor is the Student/User. Interaction happens through the main menu.

Student / User	System Use Cases
→	Manage Student Profile
→	Manage Marks
→	Check Attendance
→	Manage Study Tasks
→	Exit Program

14. Sequence Diagram – Marks Flow

This sequence shows a normal interaction when the user selects the marks manager.

Student/User	main.py	marks2.py	Console
Select option 2	Calls marks_manager()		
		Requests Maths marks	Displays prompt
Enters mark		Validates input	Receives input
Enters Python mark		Validates input	Receives input
Enters English mark		Validates input	Receives input
		Calculates total and percentage	
		Determines grade	
		Displays result	Shows result

15. Component View

The current project uses functions and modules rather than classes. A component view therefore represents the submitted design more accurately than a class diagram.

main.py
student.py | marks2.py | attendance.py | tasks.py
calculation.py | validation.py | file_manager.py
student.txt – file used by profile saving

16. Storage Design

The project uses basic file storage rather than a database. The student profile is combined into a comma-separated string and sent to the file manager for saving. The file manager can also load text and returns an empty string when the requested file is not found.

17. Design Decisions and Rationale

The main design decision is modularity. Each major activity is kept in a separate Python file. This makes the project easier for a beginner to read because profile, marks, attendance, and task code are not mixed together.

A menu-driven console was selected because it keeps attention on programming fundamentals instead of introducing a user-interface framework. The result is simple to run and easy to demonstrate.

The marks manager uses an input loop so invalid marks can be rejected without immediately ending the feature. It also handles non-numeric input. The attendance manager checks the invalid situation where attended classes are greater than total classes.

The file manager is separated from the profile module so saving and loading remain reusable operations.

18. Implementation Details

main.py imports the feature functions and displays the menu repeatedly until the user chooses exit. The marks manager processes Maths, Python, and English. The attendance manager calculates attended classes divided by total classes multiplied by 100. The task manager maintains a Python list.

The calculation module contains reusable average, percentage, and maximum functions. The validation module contains the mark-range check. These modules show how even small operations can be separated into reusable pieces.

19. Code Preservation

No changes have been made to the uploaded Python source code for this documentation. The report explains the existing implementation and keeps future ideas separate from current functionality.

20. Testing Approach

Testing can be performed directly from the console. The purpose is to verify normal behavior and the validation cases already present in the implementation.

Test	Action	Expected result
Profile	Enter name, roll number, branch, year	Profile displayed and saved.
Marks	Enter values 0–100	Total, percentage, and grade displayed.
Marks validation	Enter >100 or <0	Invalid marks message and retry.
Marks input	Enter non-numeric text	Invalid input message.
Attendance	Enter attended ≤ total	Percentage and status displayed.
Attendance validation	Enter attended > total	Invalid input message.
Tasks	Add tasks and view them	Tasks displayed with numbering.
Empty tasks	View before adding	No tasks message.
Exit	Choose option 5	Main loop ends.

21. Results

The expected result is a working terminal application in which the user can move between the four main activities from one menu. Profile, marks, and attendance provide immediate output, while the task manager maintains a small session-based task list.

22. Challenges Faced

• Dividing a student-oriented idea into manageable modules.

• Keeping marks within the valid 0–100 range.

• Handling non-numeric marks without stopping the marks workflow.

• Connecting separate modules through the main menu.

• Using file handling for profile information.

23. Learnings and Key Takeaways

This project provided practical experience with a set of Python concepts that are easy to understand individually but more meaningful when used together. A function can represent one task, modules can organize related functions, validation can make input safer, and a main menu can connect the complete application.

The project also shows that a useful beginner project does not have to be large. A clear problem, understandable workflow, modular code, and honest documentation are enough to demonstrate programming fundamentals.

24. Future Enhancements

• Add a graphical user interface.

• Use a database for structured and persistent student records.

• Add search, update, and delete operations.

• Persist study tasks between program sessions.

• Support more subjects and configurable grading rules.

• Maintain more detailed attendance records.

• Add automated unit tests.

• Use Git/GitHub history and releases more formally.

These are future possibilities only; they are not part of the current implementation.

25. References

• VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines, supplied with this submission.

• Uploaded source files: main.py, student.py, marks2.py, attendance.py, tasks.py, calculation.py, validation.py, and file_manager.py.

26. Final Note

This report has been prepared for the VITyarthi submission requirements while preserving the original coding. It documents what the project does today and clearly separates future improvements from implemented features.

Vishal Pandey
B.Tech CSE – 1st Semester
VIT Bhopal University